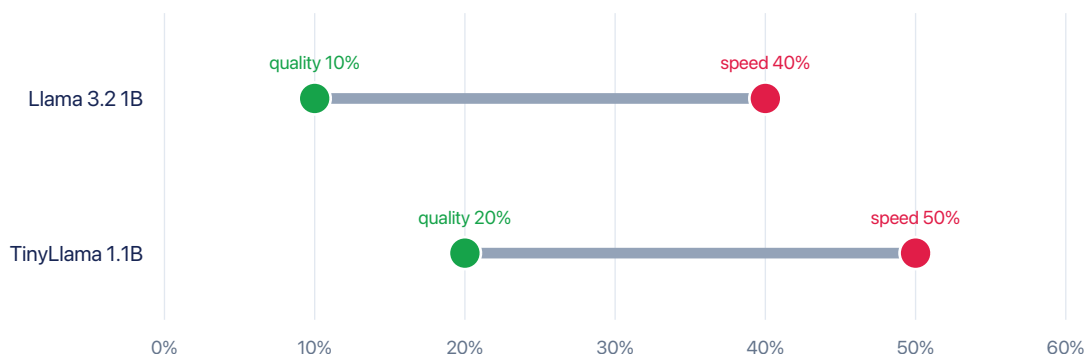


A Reproducible Investigation of the Quality–Latency Break-Even for Dynamic FFN Block Sparsity on Apple M2

Technical report • September 2026 • Sanjay Elango

BLOCKS DID ACCELERATE THE ISOLATED FFN — BUT ONLY AFTER QUALITY HAD ALREADY FAILED

No measured quality-speed intersection



Bottom line

Llama 3.2 1B preserved the screening quality gate through 10% B8 sparsity but first beat dense at 40%. TinyLlama preserved quality through 20% but first beat dense at 50%. No tested point satisfied both requirements.

This PDF is a layout-checked preview derived from committed evidence. The canonical manuscript is `paper/main.tex`.

1. Research question and contribution

Dynamic FFN sparsity promises fewer operations, but a CPU must still pay for sparse indices, fragmented weight access, branches, dispatch, cache misses, and weak vector utilization. ARM-Sparse asks whether converting token-dependent neuron selections into fixed contiguous blocks can make those skipped operations visible in wall-clock time.

The compromise

A B8 executor operates on eight adjacent intermediate neurons at once. It may calculate neurons that an ideal irregular mask would skip, but it can reuse prepacked gate/up rows and down-projection columns with regular addressing. Weight packing happens once—not once per token.

What this report contributes

Contribution	Evidence boundary
Measured quality–latency frontier	Two ~1B Llama-family models on Apple M2
Selector/executor separation	Oracle quality masks; replayed kernel timings
Strong dense comparison	Apple Accelerate FP32 dense path
Negative-result provenance	18 retained experiments; five decisive
Reproducible package	Validators, raw artifacts, figures, LaTeX, tests

Not claimed

ARM-Sparse does not invent contextual sparsity, neuron clustering, or contiguous packing. It does not demonstrate a deployable selector, end-to-end token-generation acceleration, or universal failure of block sparsity.

2. Related systems and the remaining gap

DejaVu established contextual sparsity and lightweight input-dependent predictors. LLM in a Flash showed why larger contiguous row/column transfers matter when memory movement dominates. PowerInfer separated frequently active and context-dependent neurons across CPU/GPU resources. PowerInfer-2 used neuron clusters as scheduling and storage units on smartphones. Dynamic Input Pruning combined SwiGLU pruning with cache-aware masks.

What remains worth measuring

Those systems prevent a novelty claim based merely on grouping neurons. The defensible question is narrower: at equal hardware, model geometry, and sparsity budget, where does block execution cross optimized dense latency, and does model quality survive to that same point?

Prior-work relationship

Work	Relevant idea	ARM-Sparse distinction
DejaVu	Contextual sparsity	Executor frontier, not predictor novelty
LLM in a Flash	Bundled contiguous access	RAM-resident CPU latency
PowerInfer	Hot/cold neurons	CPU-only controlled executor
PowerInfer-2	Neuron clusters	Quality–latency break-even
Dynamic Input Pruning	Cache-aware SwiGLU masks	Apple M2 reproducibility

Research policy

Hypotheses, observations, interpretations, and conclusions remain separate. Quantitative statements map to manifests; literature statements map to verified primary sources. Negative experiments remain visible.

3. Methodology and measurement boundary

A SwiGLU FFN computes $y = W_d(\text{SiLU}(W_g x) * W_u x)$, where $*$ is element-wise multiplication. The intermediate dimension is divided into contiguous groups. An active group executes its packed gate and up rows, then accumulates its packed down columns.

Quality protocol

Post-SwiGLU output-contribution oracle masks estimate the attainable quality boundary. Conditions use 16 deterministic WikiText-2 excerpts and 2,032 predicted tokens. A sparse point passes when perplexity is no more than 5% above the same-run dense condition.

Runtime protocol

Single-token synthetic FFNs match the model geometries: $2048 \times 8192 \times 2048$ and $2048 \times 5632 \times 2048$. Native packed FP32 B8 is compared with one-thread Accelerate dense. Every grid point retains 200 post-warmup measurements and is repeated across three seeds. Speedup is $100(\text{tdense}/\text{tblock} - 1)$.

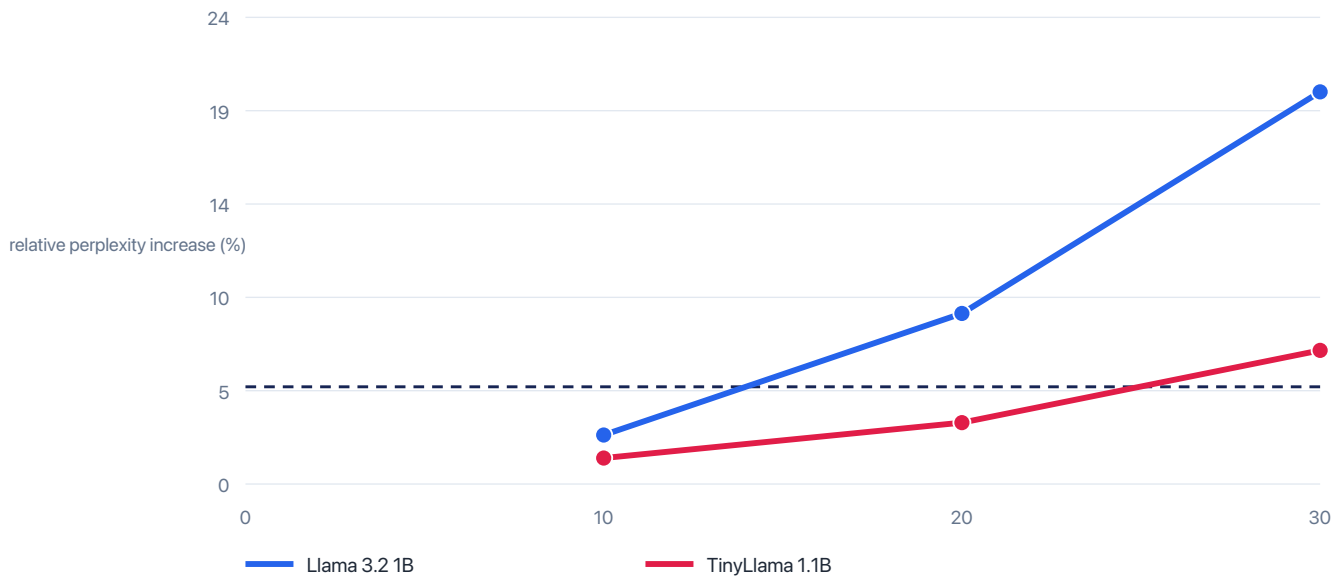
Threat-reducing choices

Choice	Why
Oracle masks	Measure attainable quality before predictor research
Replay masks	Measure executor independently from selector overhead
Optimized dense baseline	Avoid an academically weak PyTorch comparison
Discrete grid, no interpolation	Do not fabricate an exact crossover
Per-run medians and ranges	Expose repeatability without calling ranges confidence intervals

The runtime setup favors sparsity because mask selection is excluded and clustered masks reduce fragmentation. A failure here is meaningful for this executor, although it does not generalize to trained block-aware models.

4. Quality frontier

Quality cost of B8 sparsity

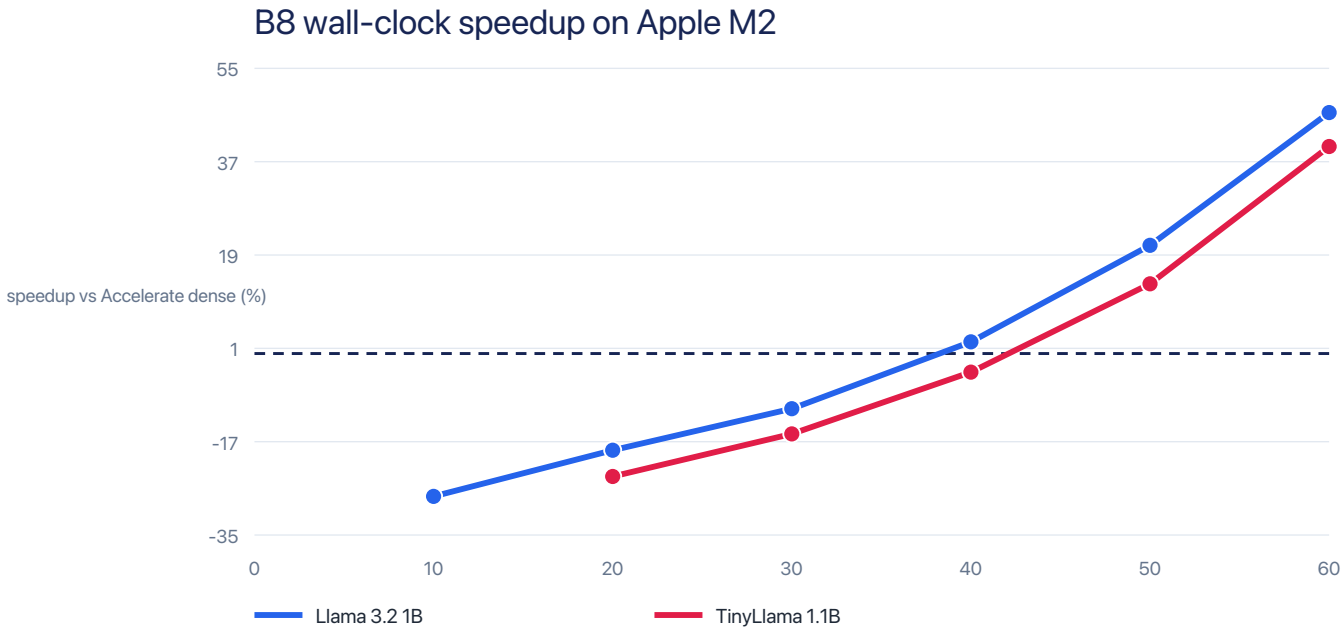


Model / condition	Relative perplexity	Gate
Llama B8 10%	+2.520%	Pass
Llama B8 20%	+8.773%	Fail
Llama B8 30%	+20.168%	Fail
TinyLlama B8 10%	+1.336%	Pass
TinyLlama B8 20%	+3.157%	Pass
TinyLlama B8 30%	+6.874%	Fail
TinyLlama B32 10%	+3.620%	Pass
TinyLlama B32 20%	+10.306%	Fail

Observation

TinyLlama tolerates more B8 sparsity than Llama, but larger B32 groups lose quality faster. Useful unstructured neuron sparsity therefore does not automatically imply useful contiguous block sparsity.

5. Runtime frontier



Geometry	Last slower point	First faster point	Speedup range
Llama 2048×8192×2048	30%	40%	+1.6 to +3.1%
TinyLlama 2048×5632×2048	40%	50%	+10.7 to +17.1%

Observation

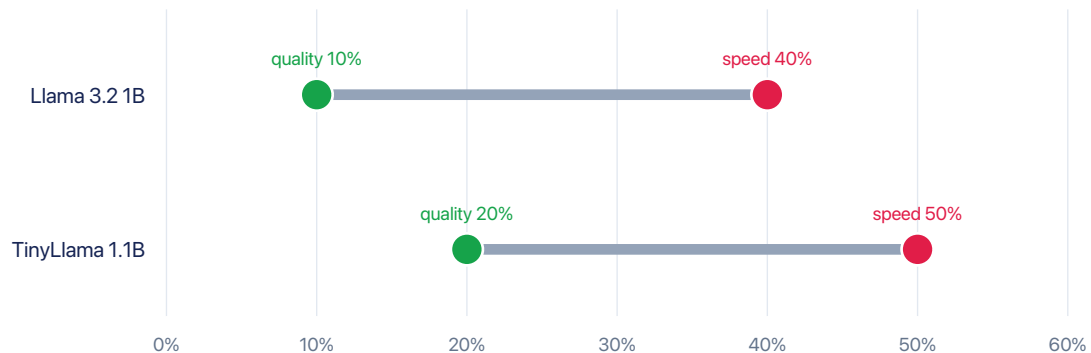
Blocks mechanically accelerate when enough groups are skipped. The smaller TinyLlama FFN needs more sparsity before overhead is amortized. At Llama's real-mask B8/10% passing point, block execution was 37.26–38.77% slower than Accelerate dense.

Interpretation

The speedup is real within the microbenchmark, but it is not usable under the chosen quality rule. Calling ARM-Sparse simply 'accelerated' would omit the most important half of the result.

6. Combined result and failed alternatives

No measured quality-speed intersection



Five decisive experiments

ID	Role	Outcome
EXP-004	Llama quality	B8 ceiling: 10%
EXP-005	Passing-mask replay	37–39% slower
EXP-012	Llama runtime	First win: 40%
EXP-016	TinyLlama quality	B8 ceiling: 20%
EXP-018	TinyLlama runtime	First win: 50%

Why the other 13 experiments remain

They establish baselines, test mask expansion and scoring, evaluate neuron reordering and hot/cold layouts, profile phases, try explicit NEON and coalescing, and probe INT8/B32 execution. These attempts did not produce a quality-preserving win. Keeping them prevents cherry-picking; excluding them from the headline prevents an unreadable 18-experiment narrative.

Quantization boundary

The production llama.cpp Q8_0 result is a dense context baseline. The minimal B32 Q8_0 executor did not reproducibly beat its matched dense control through 80% sparsity. Custom INT8 wins against a weak custom dense control do not supersede the optimized Accelerate result.

7. Conclusion, next decision, and reproducibility

ARM-Sparse completed its bounded investigation. Contiguous blocks can accelerate isolated FFN execution, but the tested models lose acceptable quality before enough blocks can be skipped. Llama's quality ceiling and first speed point are 10% and 40%; TinyLlama's are 20% and 50%. No measured point satisfies both.

What should happen next

Do not continue tuning the same executor without a mechanism capable of moving the quality frontier by roughly 30 percentage points. A scientifically stronger continuation would train for block-aligned activation, jointly learn selector and layout, or evaluate a model architecture with native grouped structure. Each should have a preregistered stop condition.

One-command audit

After installing the declared development dependencies, run `make verify-package`. It validates core manifests, checks deterministic package artifacts, and runs the full test suite. `make package` regenerates the machine-readable summary, three SVG figures, and LaTeX table without touching raw results.

Evidence map

Artifact	Purpose
research/claims.yaml	Canonical quantitative claims and limitations
research/package/summary.json	Generated figure/table data
research/package/claim-audit.md	Claim and citation mapping
paper/main.tex	Canonical manuscript
docs/article.md	Public technical narrative
REPRODUCING.md	Environment and command boundary

Final answer

Under the recorded Apple M2 setup, block-based dynamic FFN sparsity worked as a high-sparsity kernel optimization, but did not work as a quality-preserving LLM inference method.

Primary literature: Liu et al. (ICML 2023); Alizadeh et al. (ACL 2024); Song et al. (SOSP 2024); Xue et al. (2024); Sudarshan et al. (MLSys 2025). Full references are in `paper/references.bib`.